
Documentação de Projeto

para o sistema

CoinPrimer

Versão 1.0

Projeto de sistema elaborado pelo aluno Matheus Felipe
como parte da disciplina **Projeto de Software**.

13/05/2026

Tabela de Conteúdo

1. Introdução	1
2. Modelos de Usuário e Requisitos	1
2.1 Descrição de Atores	1
2.2 Modelo de Casos de Uso e Histórias de Usuários	1
2.3 Diagrama de Sequência do Sistema e Contrato de Operações	1
3. Modelos de Projeto	1
3.1 Arquitetura	1
3.2 Diagrama de Componentes e Implantação.	2
3.3 Diagrama de Classes	2
3.4 Diagramas de Sequência	2
3.5 Diagramas de Comunicação	2
3.6 Diagramas de Estados	2
4. Modelos de Dados	2

Histórico de Revisões

Nome	Data	Razões para Mudança	Versão
Matheus Felipe	13/05/2026	Criação inicial do documento com diagramas UML (casos de uso, classes, componentes, sequência, estados, comunicação) e modelagem de dados.	1.0

1. Introdução

Este documento agrega: a elaboração e revisão de modelos de domínio e modelos de projeto para o sistema **CoinPremier**. A referência principal para a descrição geral do problema, domínio e requisitos do sistema é o documento de especificação que descreve a visão de domínio do sistema.

O **CoinPremier** é um sistema de moeda virtual acadêmica que permite aos professores reconhecerem seus alunos através de moedas digitais, que podem ser trocadas por produtos e descontos oferecidos por empresas parceiras.

Objetivos do sistema:

- Incentivar o bom desempenho acadêmico e o engajamento dos alunos
- Fortalecer a parceria entre instituições de ensino e empresas
- Gamificar a experiência universitária
- Reconhecer publicamente o mérito estudantil

Domínio do sistema: O sistema envolve 4 tipos de atores (Aluno, Professor, Empresa Parceira e Administrador), vinculados a instituições de ensino, onde professores recebem 1.000 moedas virtuais por semestre e distribuem aos alunos como reconhecimento. Os alunos acumulam essas moedas e podem trocá-las por vantagens reais (produtos, descontos) oferecidas por empresas parceiras através de uma lojinha virtual.

2. Modelos de Usuário e Requisitos

2.1 Descrição de Atores

O sistema CoinPremier envolve quatro atores humanos e um ator sistema automático:

2.1.1 Admin (Administrador)

- **Descrição:** Usuário com permissões totais sobre o sistema. Responsável pela gestão operacional.
- **Função principal:** Gerenciar instituições, cadastrar professores, gerenciar empresas parceiras, definir categorias de vantagens e auditar o sistema.
- **Relacionamento com o sistema:** Acesso privilegiado a todas as entidades.

2.1.2 Aluno

- **Descrição:** Estudante vinculado a uma instituição de ensino parceira.
- **Função principal:** Receber reconhecimentos (moedas) dos professores, navegar na lojinha virtual, favoritar, adicionar ao carrinho e resgatar vantagens.
- **Relacionamento com o sistema:** Auto-cadastrado; possui saldo acumulativo.

2.1.3 Professor

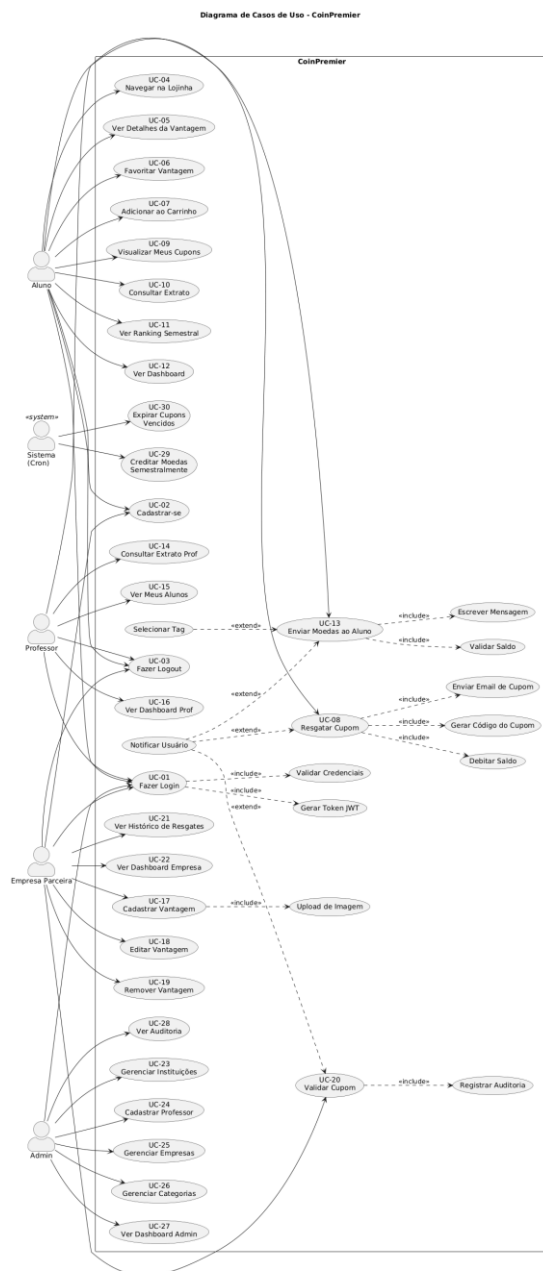
- **Descrição:** Docente pré-cadastrado pelo admin, vinculado a uma instituição de ensino.
- **Função principal:** Distribuir moedas aos alunos como reconhecimento por mérito acadêmico, participação, criatividade, liderança etc.
- **Relacionamento com o sistema:** Recebe 1.000 moedas automaticamente a cada semestre (acumuláveis).

2.1.4 Empresa Parceira

- **Descrição:** Empresa que oferece vantagens (produtos, descontos) aos alunos.
- **Função principal:** Cadastrar vantagens, receber notificações de resgates e validar cupons apresentados presencialmente pelos alunos.
- **Relacionamento com o sistema:** Auto-cadastrada; gerencia seu próprio catálogo.

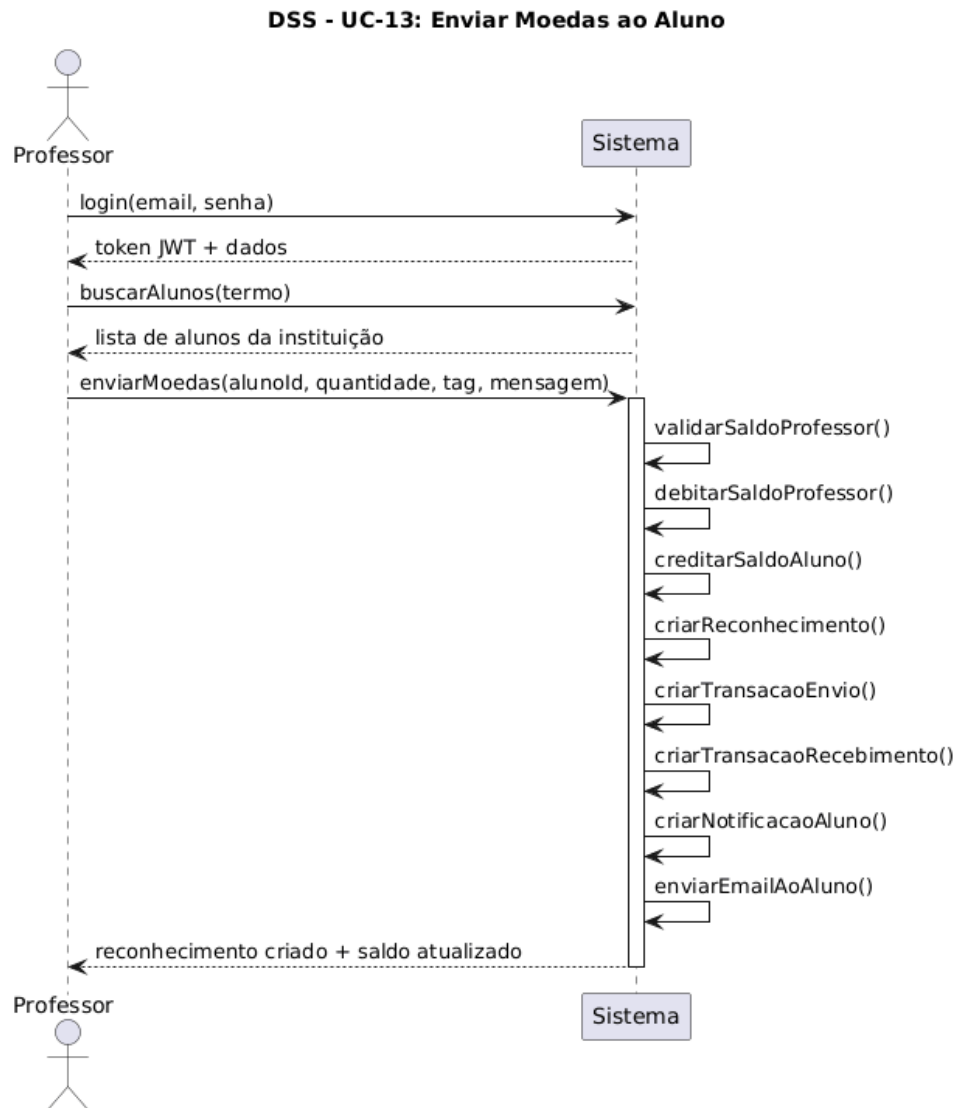
2.1.5 Sistema (Ator Automático)

- **Descrição:** Processos automáticos executados pelo sistema sem intervenção humana.
- **Função principal:** Executar jobs agendados, como crédito semestral de moedas (dia 1º do primeiro mês de cada semestre) e expiração automática de cupons vencidos.
- **Relacionamento com o sistema:** Cron jobs agendados via node-cron.



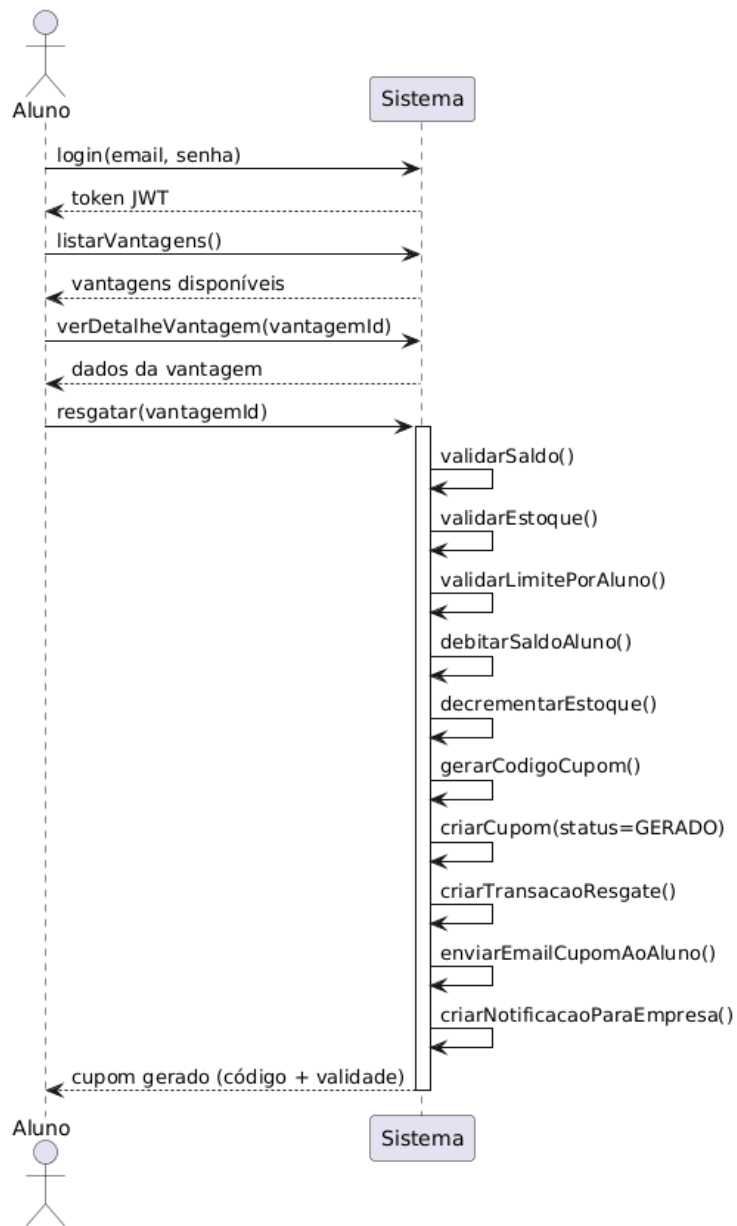
2.3 Diagrama de Sequência do Sistema

DSS-01: Enviar Moedas (UC-13)	
Contrato	CO-01
Operação	enviarMoedas(professorId, alunoId, quantidade, tag, mensagem)
Referências cruzadas	UC-13 (Enviar Moedas)
Pré-condições	1. Professor autenticado com role = PROFESSOR 2. Aluno existe e pertence à mesma instituição do professor 3. Saldo do professor $\geq$ quantidade 4. Quantidade é inteiro positivo ≥ 1 5. Mensagem possui entre 10 e 500 caracteres 6. Tag pertence ao enum TagReconhecimento
Pós-condições	1. Professor.saldoMoedas foi decrementado em quantidade 2. Aluno.saldoMoedas foi incrementado em quantidade 3. Uma instância de Reconhecimento foi criada 4. Duas instâncias de Transação foram criadas (ENVIO e RECEBIMENTO) 5. Uma Notificação foi criada para o aluno 6. Um e-mail foi disparado ao aluno 7. Toda a operação foi executada atomicamente (tudo ou nada)



DSS-02: Resgatar Cupom (UC-08)	
Contrato	CO-02
Operação	resgatarVantagem(alunoId, vantagemId)
Referências cruzadas	UC-08 (Resgatar Cupom)
Pré-condições	1. Aluno autenticado com role = ALUNO 2. Vantagem existe e está com ativo = true 3. Saldo do aluno $\geq$ custoMoedas da vantagem 4. Estoque da vantagem > 0 (ou ilimitado/null) 5. Aluno não atingiu limitePorAluno para esta vantagem
Pós-condições	1. Aluno.saldoMoedas foi decrementado em custoMoedas 2. Vantagem.estoque foi decrementado em 1 (se não for ilimitado) 3. Uma instância de Cupom foi criada com status = GERADO, código único e custoMoedasSnapshot travado 4. Cupom.dataValidade foi calculada como now() + validadeCupomDias 5. Uma instância de Transacao foi criada (RESGATE) 6. Um email com o cupom foi enviado ao aluno 7. Uma Notificacao foi criada para a empresa 8. Toda a operação foi executada atomicamente

DSS - UC-08: Resgatar Cupom



DSS-03: Validar Cupom (UC-20)	
Contrato	CO-03
Operação	validarCupom(empresaId, codigoCupom)
Referências cruzadas	UC-20 (Validar Cupom)
Pré-condições	1. Empresa autenticada com role = EMPRESA 2. Cupom existe com o código informado 3. Cupom.status = GERADO 4. Cupom.dataValidade > now() 5. Cupom pertence a uma vantagem da empresa logada (Cupom.vantagem.empresaId = empresa.id)
Pós-condições	1. Cupom.status foi alterado para UTILIZADO 2. Cupom.dataUtilizacao foi preenchido com now() 3. Cupom.validadoPorEmpresaId recebeu o id da empresa validadora (auditoria) 4. Uma Notificacao foi criada para o aluno5. Um registro de auditoria foi gerado



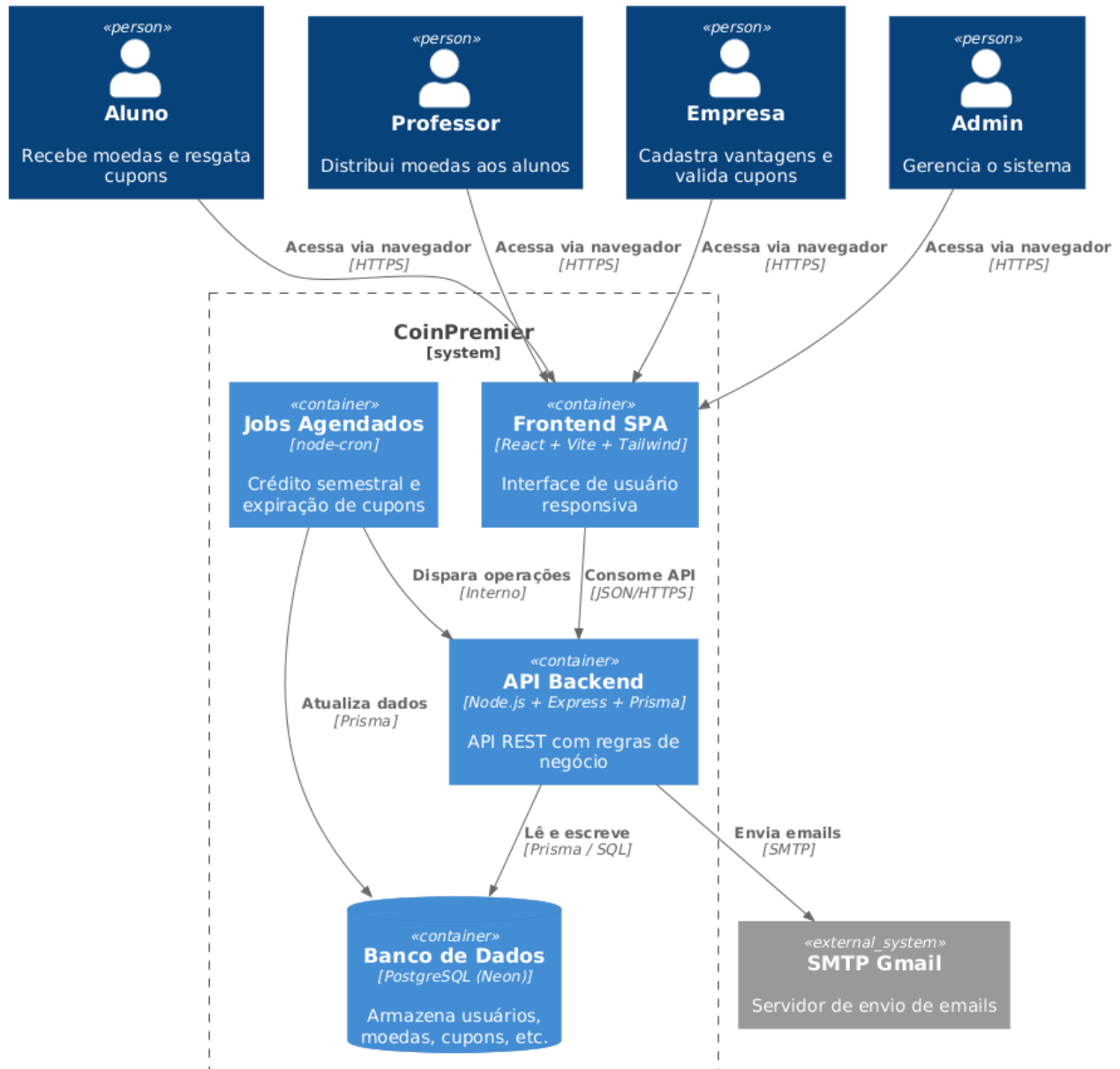
3. Modelos de Projeto

3.1 Arquitetura

O CoinPrimer segue arquitetura em camadas (MVC) no backend e Component-Based no frontend, com separação clara de responsabilidades:

- **Frontend:** React + Vite + Tailwind CSS + Zustand
- **Backend:** Node.js + Express + Prisma ORM (MVC + Repository Pattern)
- **Banco:** PostgreSQL hospedado no Neon Database
- **Comunicação:** API REST com autenticação JWT
- **Emails:** SMTP via Nodemailer

Diagrama de Containers (C4) - CoinPremier



3.2 Diagrama de Componentes e Implantação.

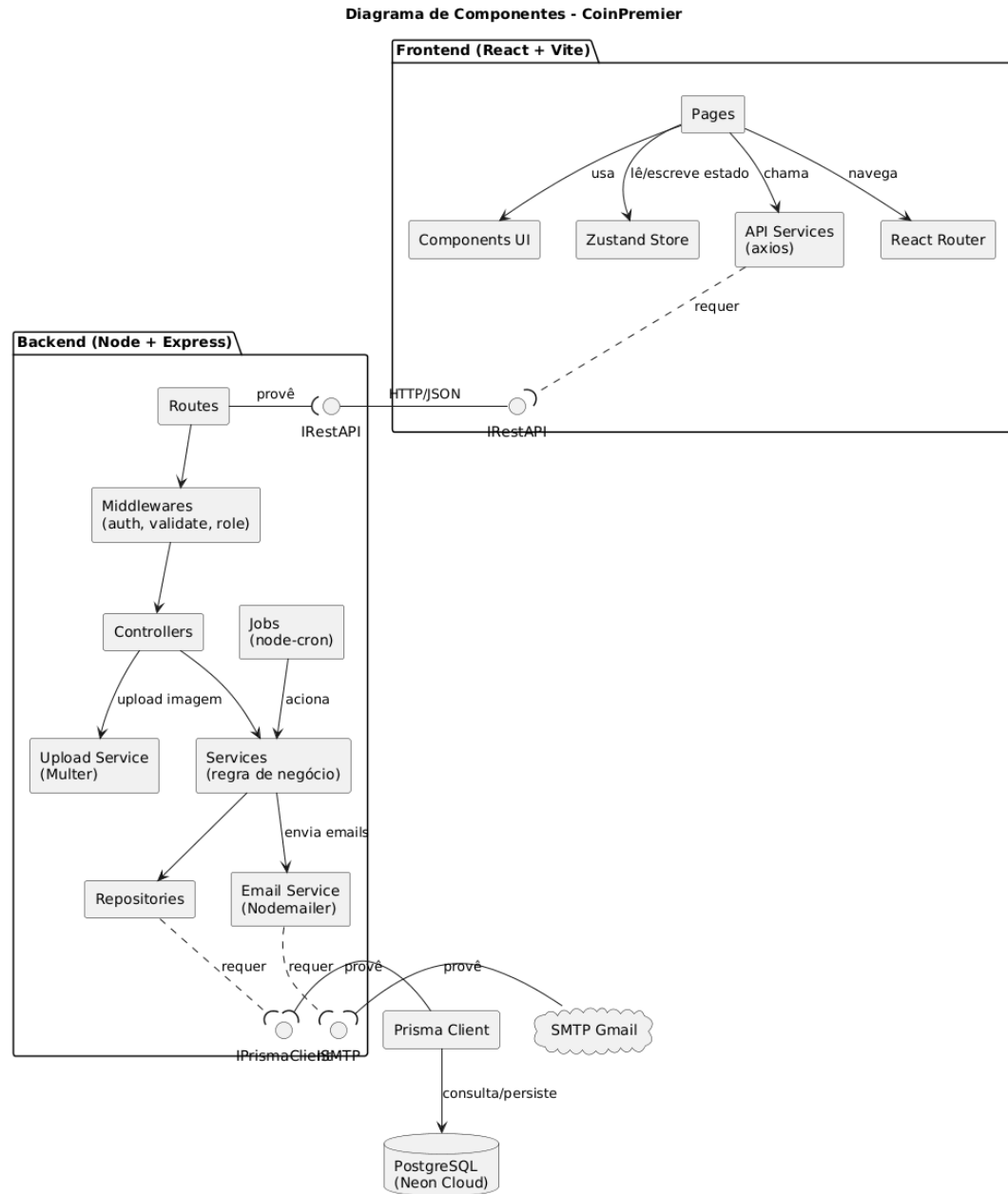
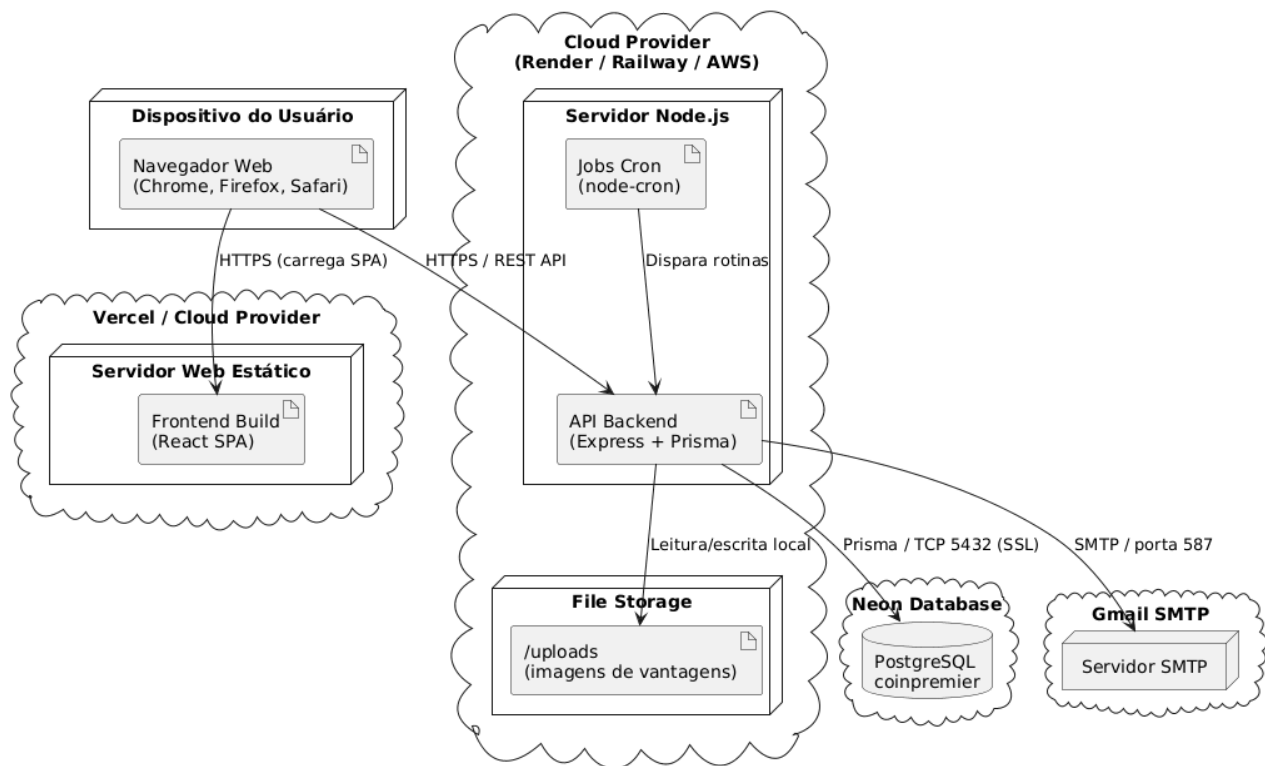
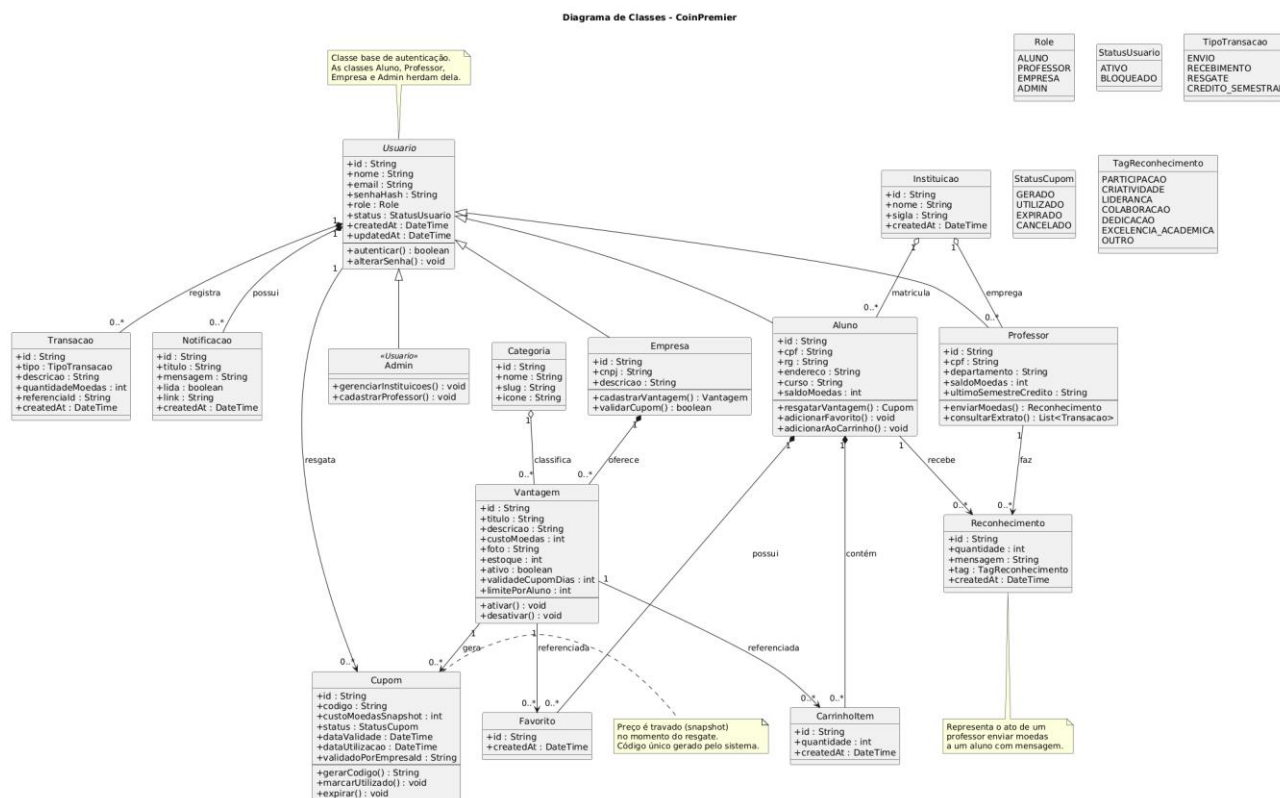


Diagrama de Implantação - CoinPrimer

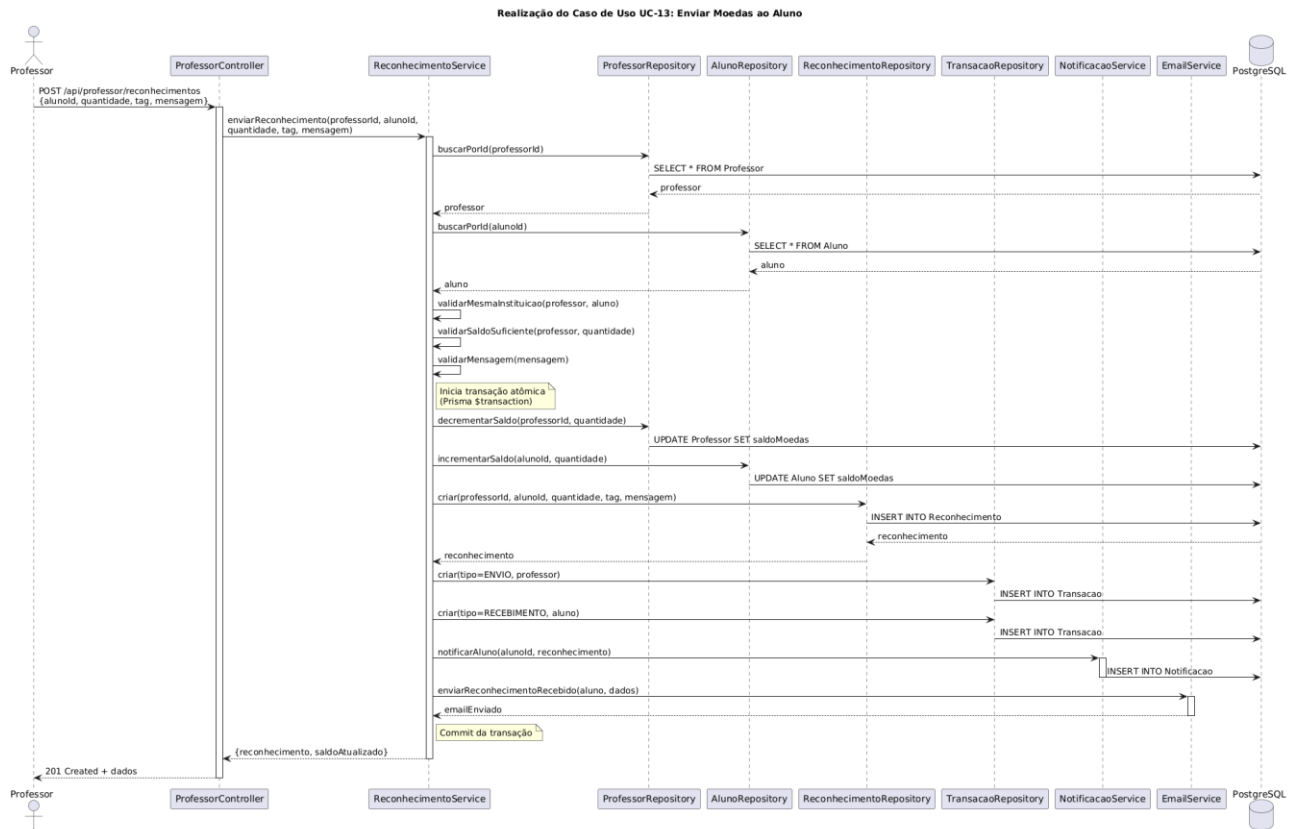


3.3 Diagrama de Classes

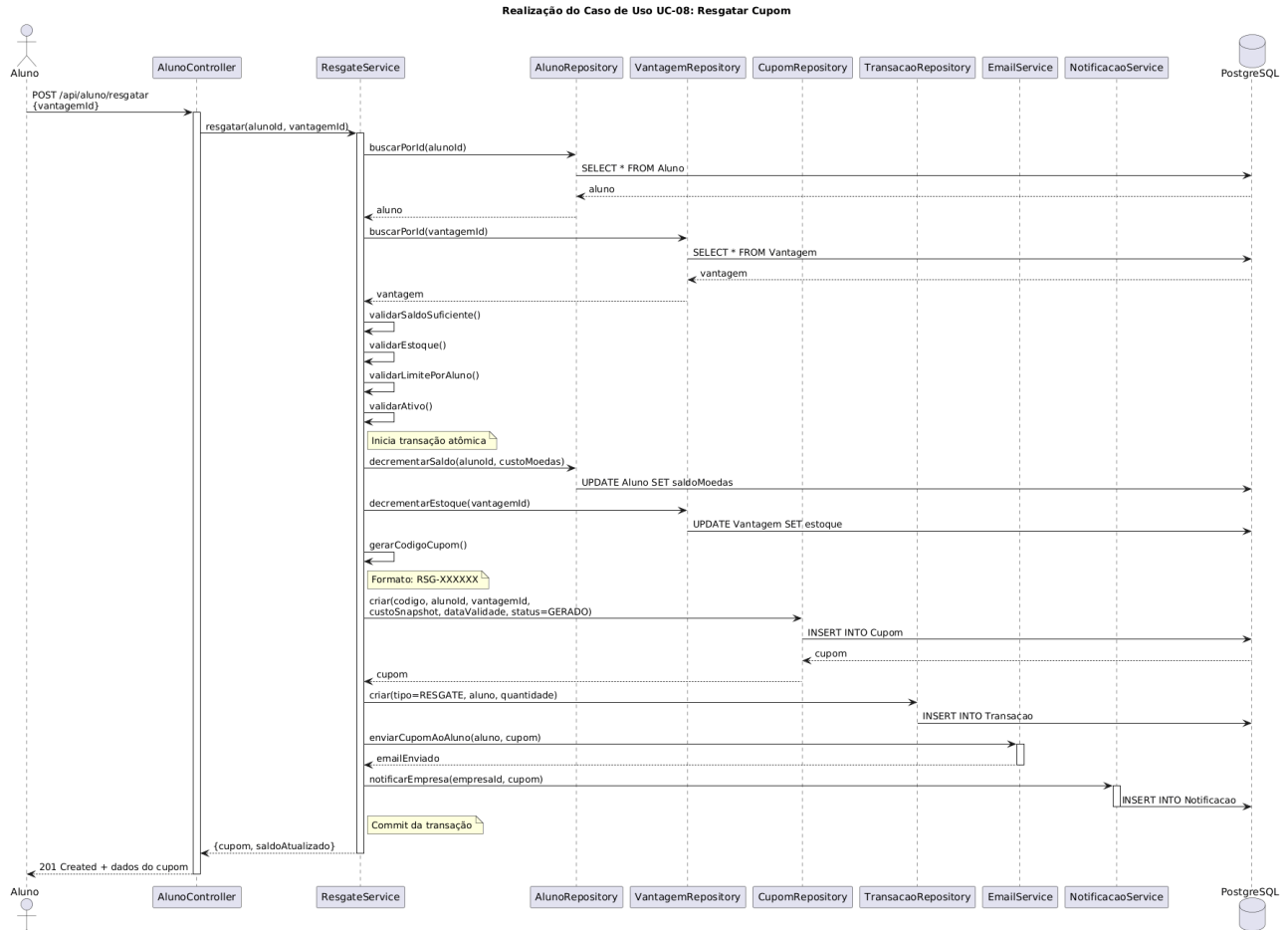


3.4 Diagramas de Sequência

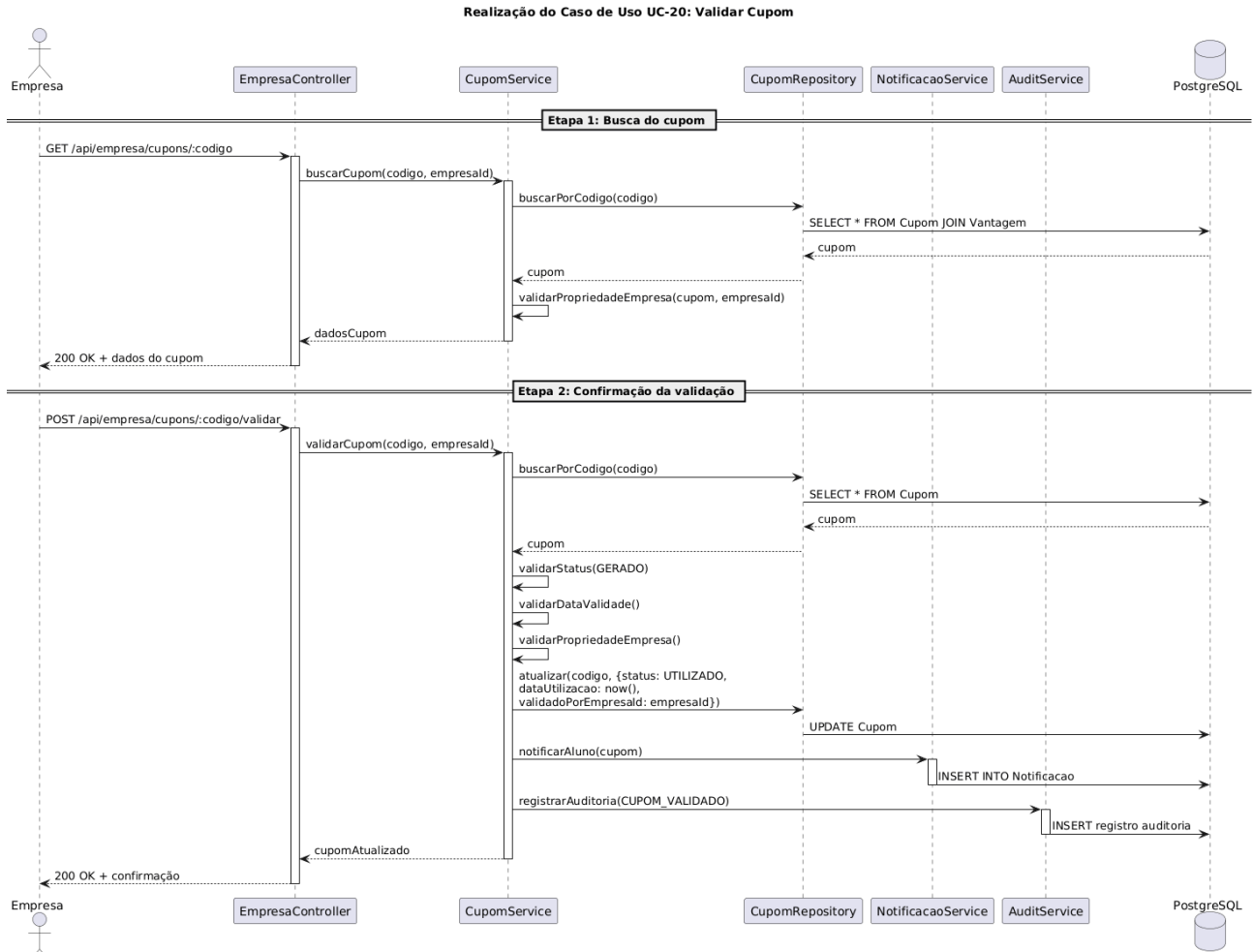
3.4.1 DS-01: Realização do UC-13 (Enviar Moedas ao Aluno)



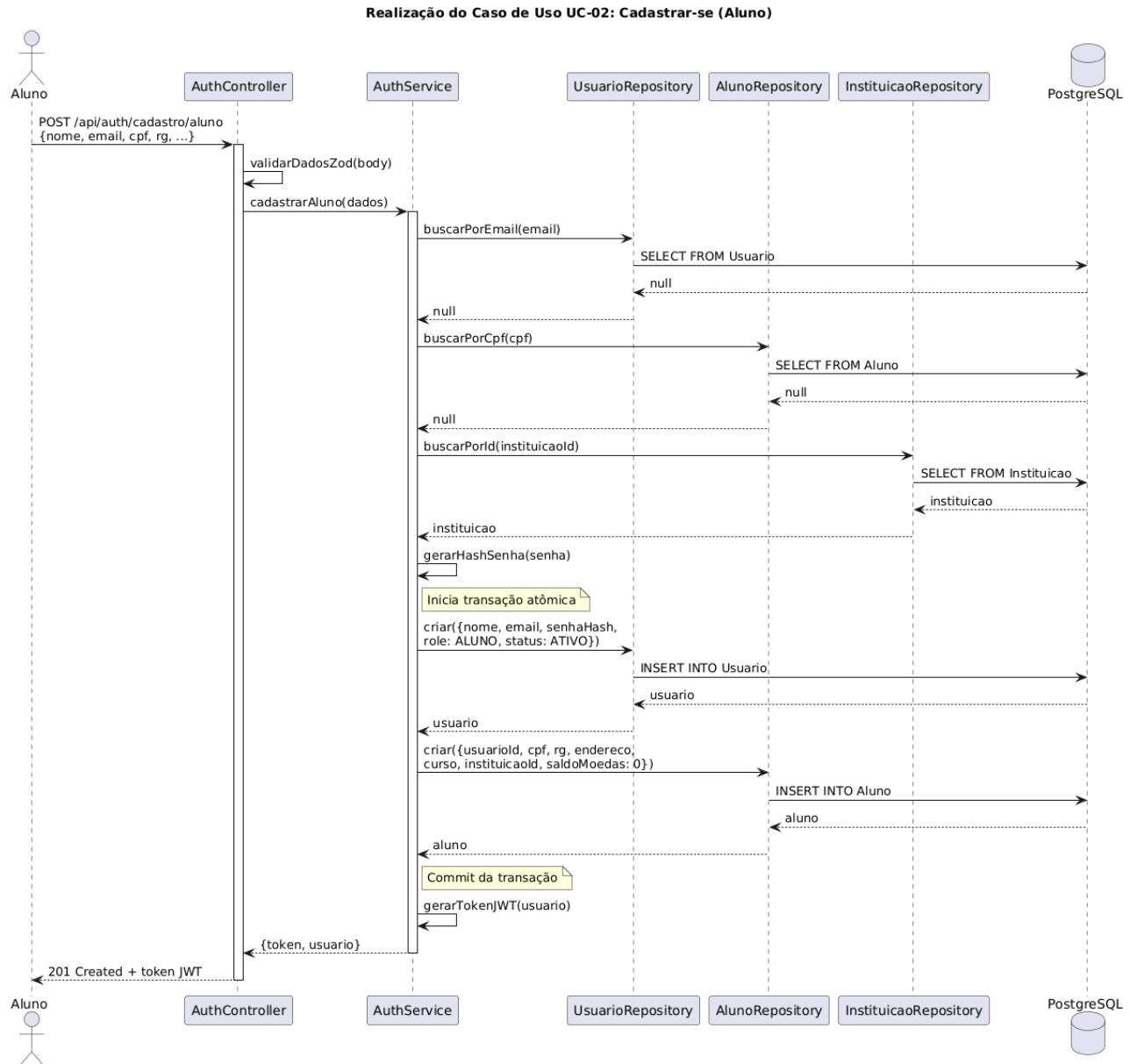
3.4.2 DS-02: Realização do UC-08 (Resgatar Cupom)



3.4.3 DS-03: Realização do UC-20 (Validar Cupom)



3.4.4 DS-04: Realização do UC-02 (Cadastrar Aluno)

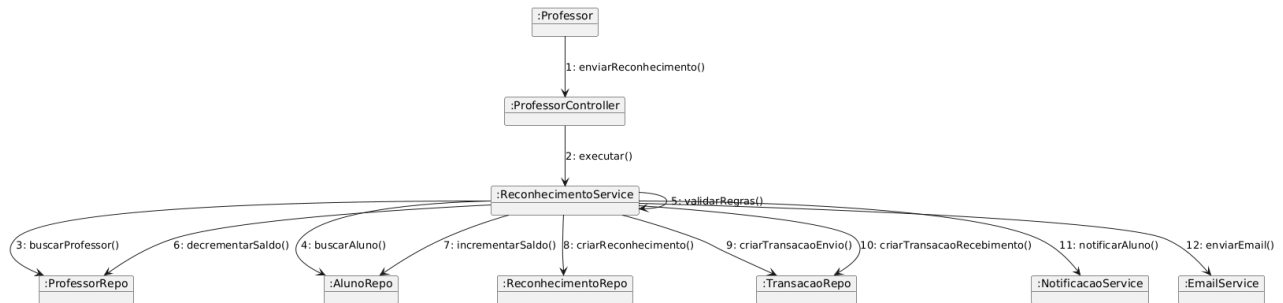


3.5 Diagramas de Comunicação

Os diagramas de comunicação representam as mesmas interações dos diagramas de sequência, mas com ênfase nas ligações entre objetos (visão estrutural).

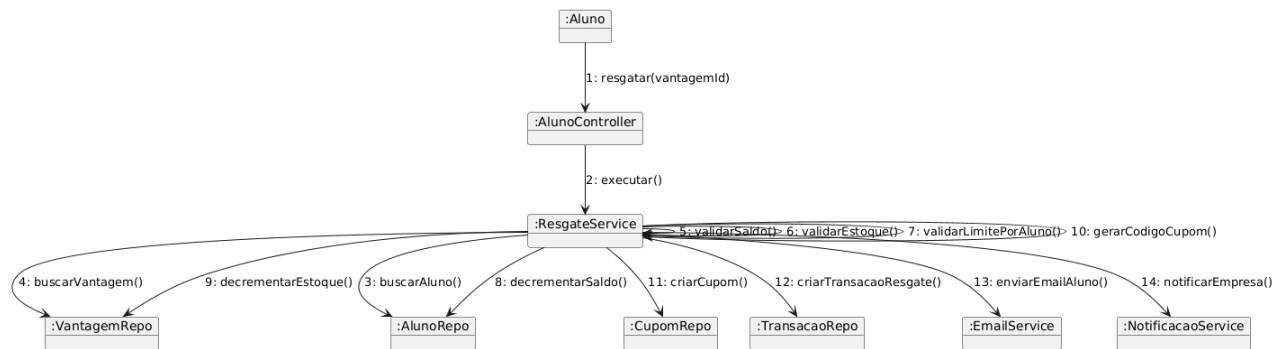
3.5.1 DC-01: Realização do UC-13 (Enviar Moedas)

Diagrama de Comunicação - UC-13 Enviar Moedas



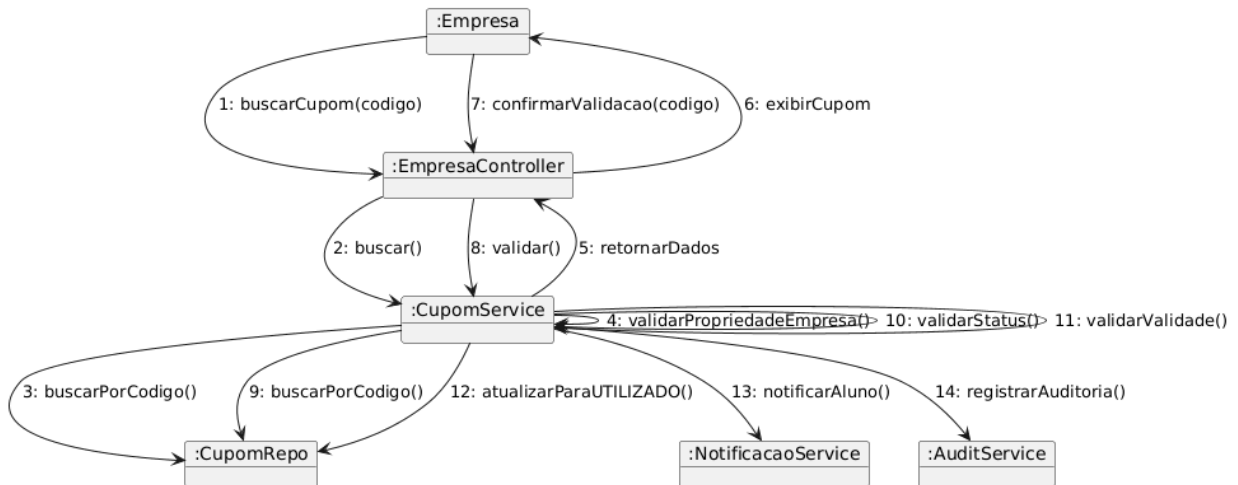
3.5.2 DC-02: Realização do UC-08 (Resgatar Cupom)

Diagrama de Comunicação - UC-08 Resgatar Cupom



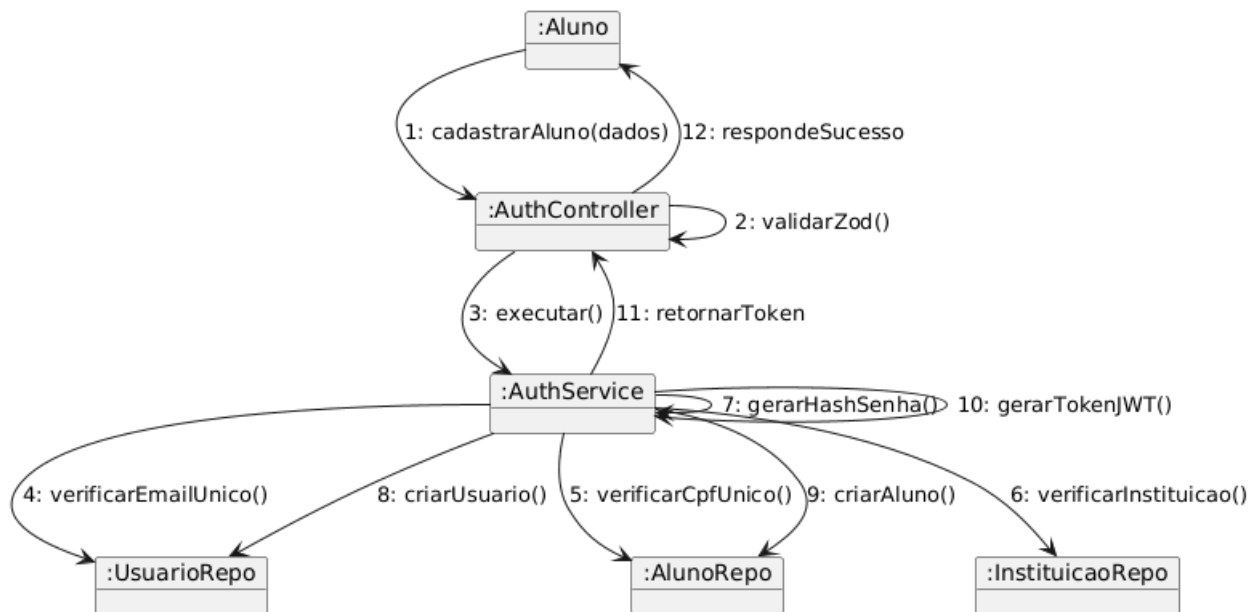
3.5.3 DC-03: Realização do UC-20 (Validar Cupom)

Diagrama de Comunicação - UC-20 Validar Cupom



3.5.4 DC-04: Realização do UC-02 (Cadastrar Aluno)

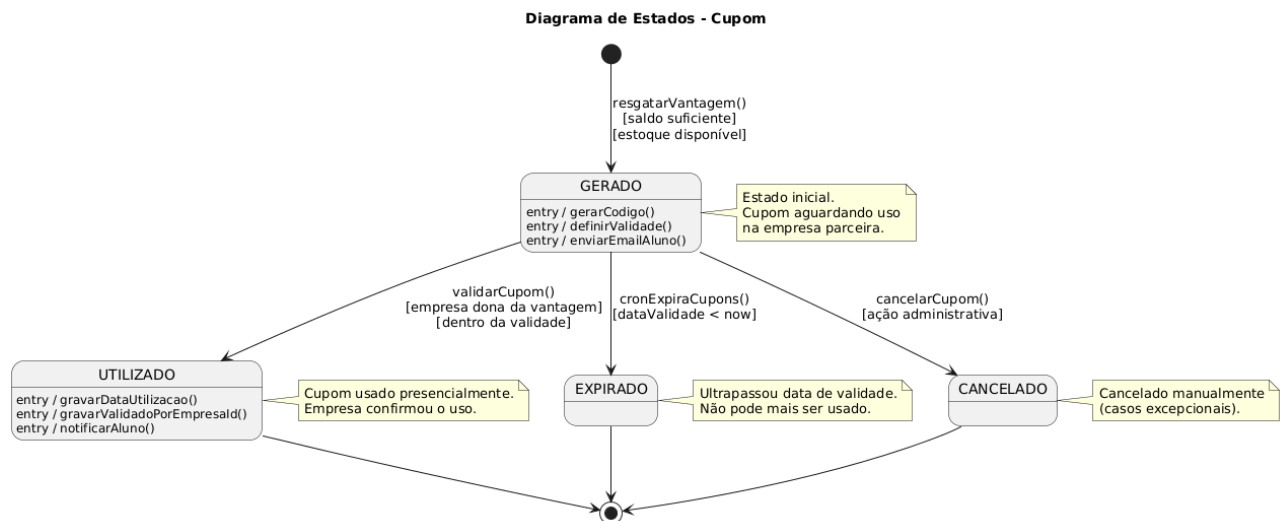
Diagrama de Comunicação - UC-02 Cadastrar-se (Aluno)



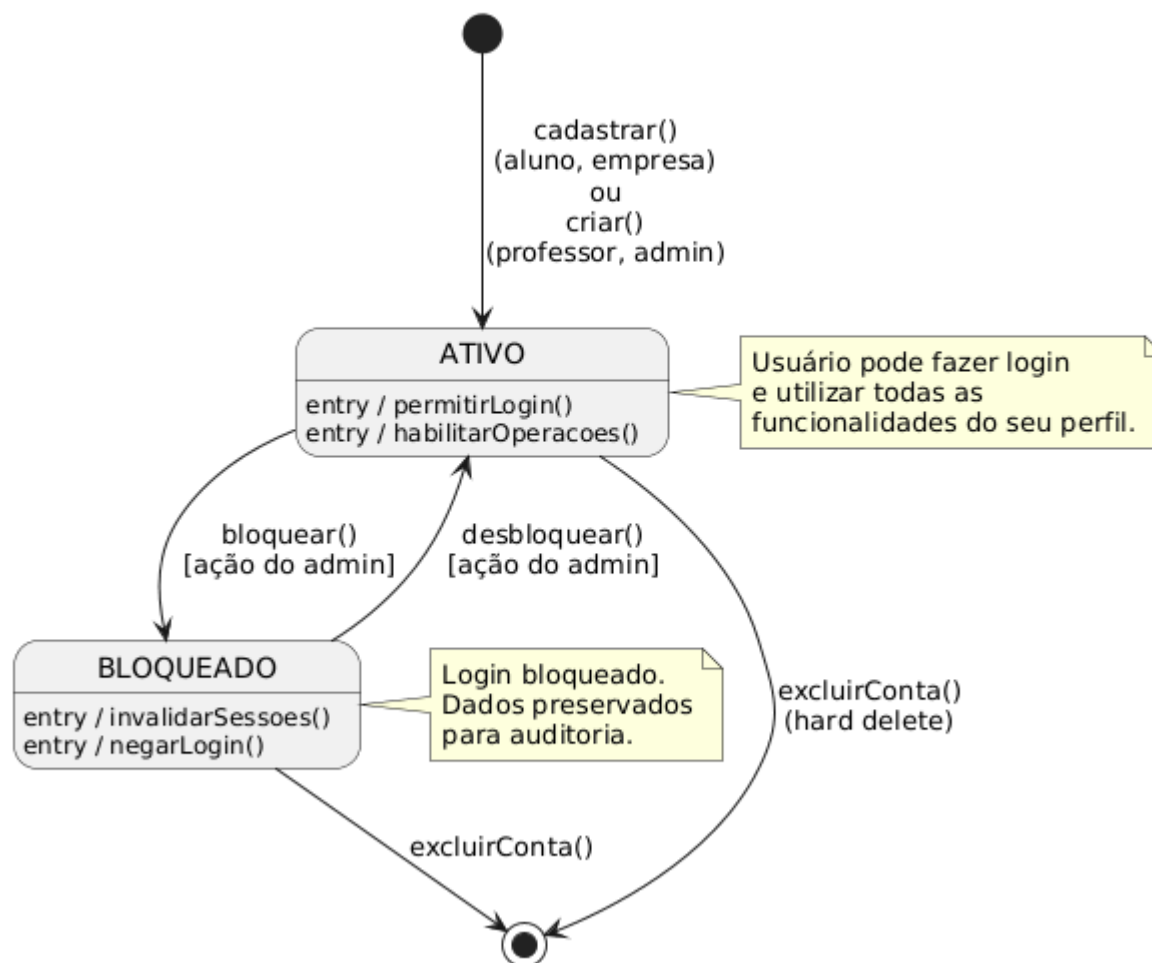
3.6 Diagramas de Estados

Nesta subseção são apresentados os diagramas de estados das principais entidades do sistema, mostrando suas transições ao longo do ciclo de vida.

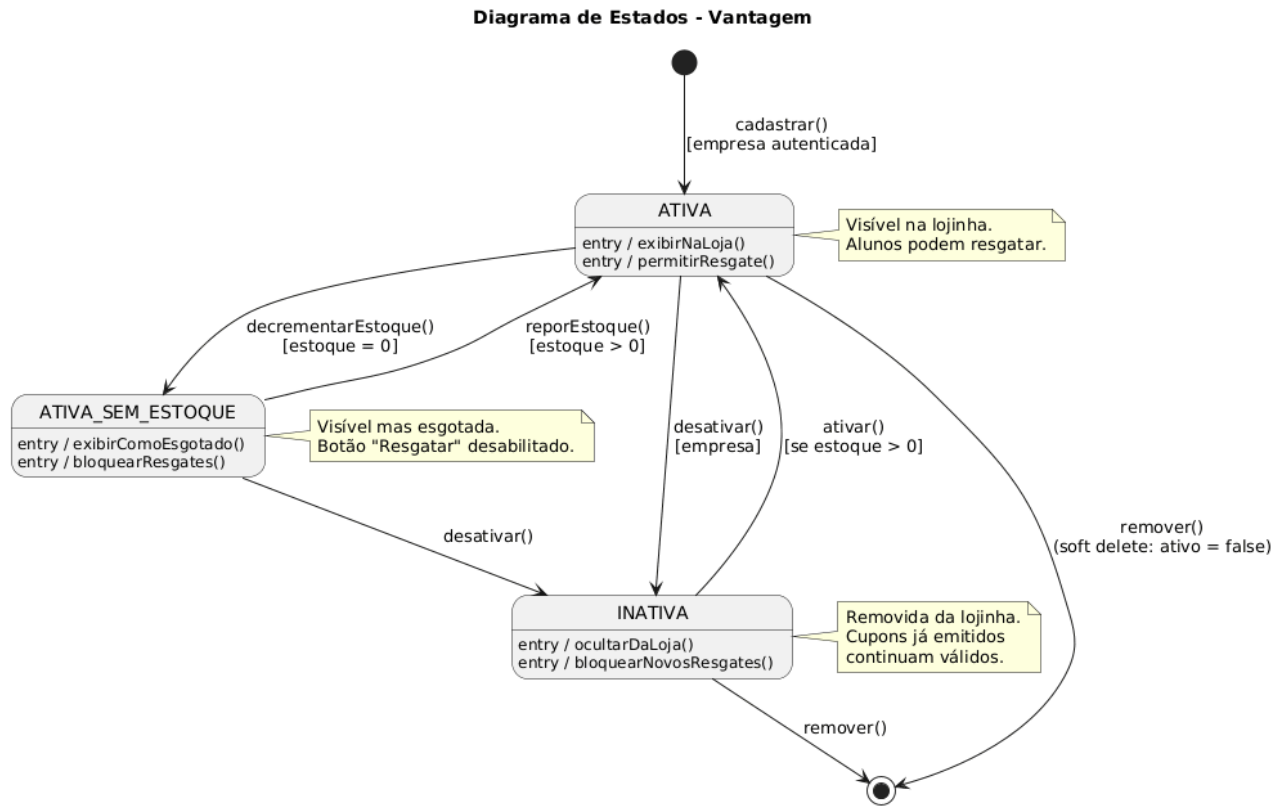
3.6.1 DE-01: Estados do Cupom



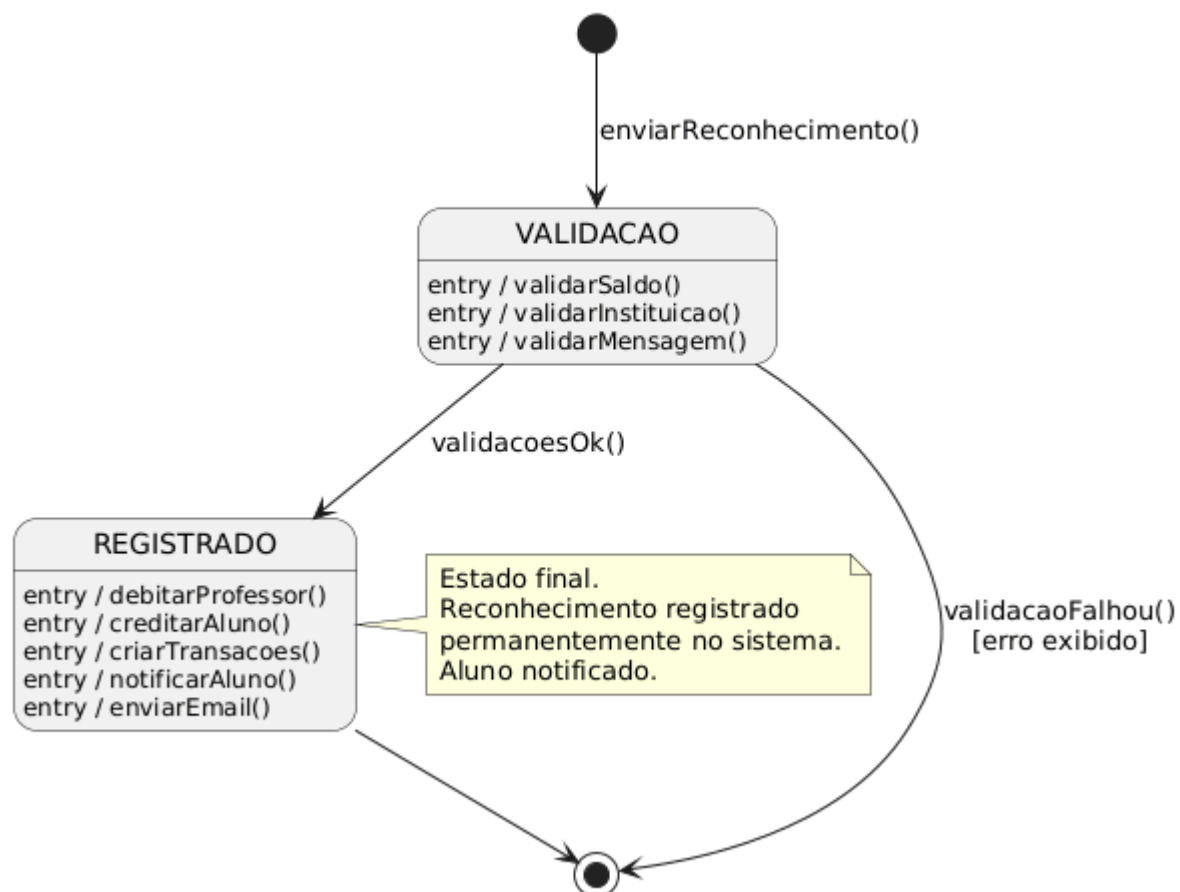
3.6.2 DE-02: Estados do Usuário

Diagrama de Estados - Usuário

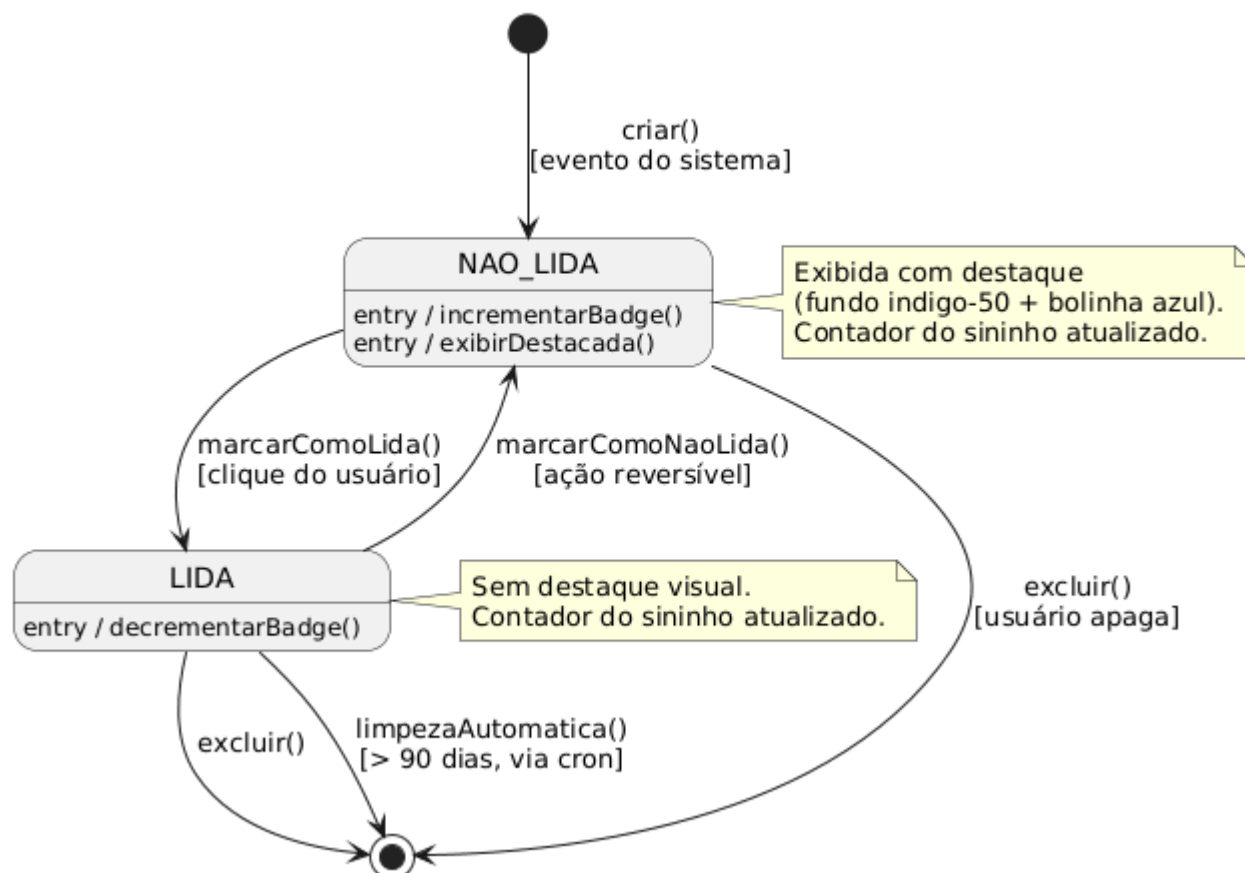
3.6.3 DE-03: Estados da Vantagem



3.6.4 DE-04: Estados do Reconhecimento (simplificado)

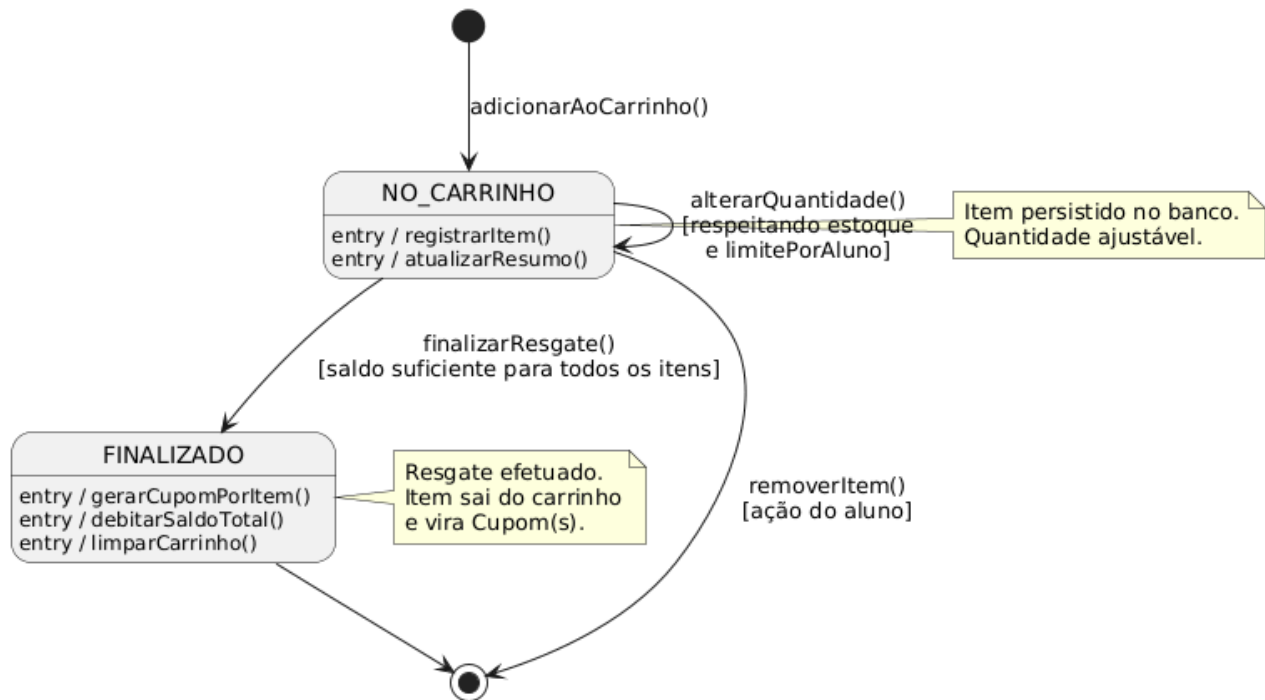
Diagrama de Estados - Reconhecimento

3.6.5 DE-05: Estados da Notificação

Diagrama de Estados - Notificação

3.6.6 DE-06: Estados do Item do Carrinho

Diagrama de Estados - Item do Carrinho



4. Modelos de Dados

4.1 Estratégia de Mapeamento Objeto-Relacional

O CoinPremier utiliza a estratégia **ORM + Repository Pattern** para acesso ao banco de dados, implementada com **Prisma ORM** sobre **PostgreSQL** (hospedado no Neon Cloud).

Critério	Decisão	Justificativa
ORM	Prisma Client	Type-safety nativo, migrations automáticas, schema declarativo, excelente suporte ao PostgreSQL
Padrão de Acesso	Repository Pattern	Isola Services (lógica de negócio) da camada de persistência, facilita testes unitários (mocks) e permite futura troca de ORM
Transações Atômicas	prisma.\$transaction()	Garante ACID em operações críticas (resgate, envio de moedas)
IDs	CUID (@default(cuid()))	IDs únicos, ordenáveis e resistentes a colisão em sistemas distribuídos
Soft Delete	Flag ativo: boolean	Preserva integridade referencial (cupons emitidos permanecem válidos mesmo se vantagem for "removida")

4.1.2 Fluxo de Acesso aos Dados

Controller → Service → Repository → Prisma Client → PostgreSQL

- **Controller** recebe requisição HTTP e delega ao Service
- **Service** aplica regras de negócio e chama um ou mais Repositories
- **Repository** encapsula queries Prisma (nunca usado diretamente pelo Controller)
- **Prisma Client** traduz chamadas em SQL e gerencia conexões
- **PostgreSQL** persiste os dados

4.1.3 Técnicas Especiais de Mapeamento

Herança (Usuario → Aluno/Professor/Empresa): Modelada como **composição 1:1** no banco relacional, não como tabela única. Cada perfil tem sua tabela com chave estrangeira `usuarioId @unique`. A distinção entre perfis é feita pelo campo `role` em `Usuario`. Esta estratégia é conhecida como **Class Table Inheritance**.

Relacionamentos N:N: Implementados por tabelas associativas com índices únicos compostos:

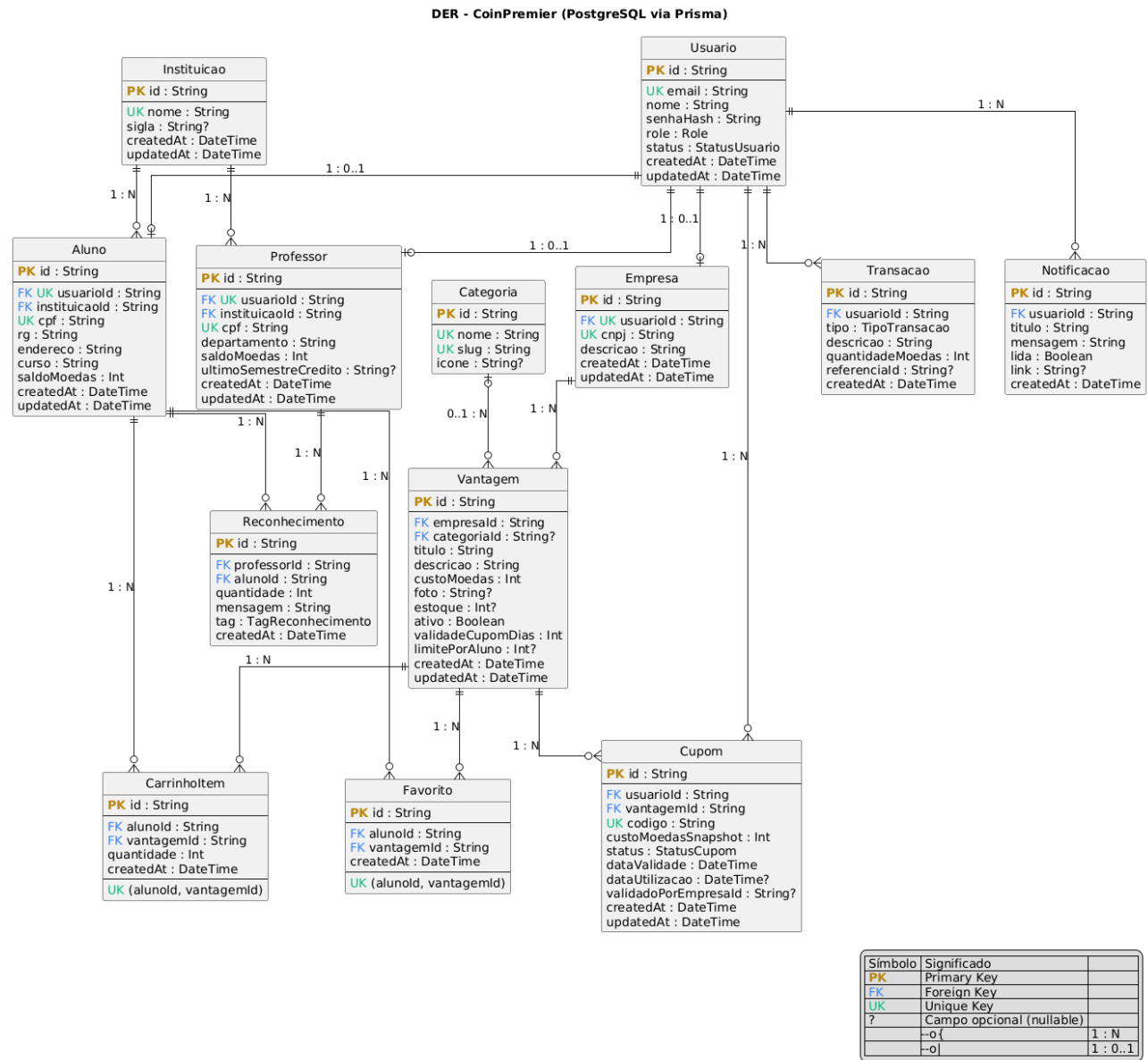
- Favorito (`alunoId`, `vantagemId`) — único composto
- CarrinhoItem (`alunoId`, `vantagemId`) — único composto
- Cupom e Reconhecimento — não usam `unique` composto pois podem ter múltiplos registros entre os mesmos pares

Snapshot de Valores: O campo `custoMoedasSnapshot` em `Cupom` guarda o preço no instante do resgate, garantindo imutabilidade histórica. Se a empresa alterar o preço da `Vantagem` posteriormente, cupons já emitidos mantêm o valor original.

Índices Compostos: Aplicados em consultas frequentes para otimização:

- `Transacao @@index([usuarioId, createdAt])` — extrato cronológico
- `Cupom @@index([status, dataValidade])` — expiração via cron
- `Reconhecimento @@index([alunoId, createdAt])` — histórico do aluno

4.2 Diagrama Entidade-Relacionamento (DER)



4.3 Esquema Físico do Banco (DDL Simplificado)

O script DDL abaixo representa a estrutura física gerada pelo Prisma Migrate. Apresentado em formato simplificado para fins de documentação.

```
sql
```

```
--
--
CREATE TYPE "Role" AS ENUM ('ALUNO', 'PROFESSOR', 'EMPRESA', 'ADMIN');
CREATE TYPE "StatusUsuario" AS ENUM ('ATIVO', 'BLOQUEADO');
CREATE TYPE "TipoTransacao" AS ENUM ('ENVIO', 'RECEBIMENTO', 'RESGATE', 'CREDITO_SEMESTRAL');
CREATE TYPE "StatusCupom" AS ENUM ('GERADO', 'UTILIZADO', 'EXPIRADO', 'CANCELADO');
CREATE TYPE "TagReconhecimento" AS ENUM (
```

```

'PARTICIPACAO',          'CRIATIVIDADE',          'LIDERANCA',          'COLABORACAO',
'DEDICACAO',              'EXCELENCIA_ACADEMICA',          'OUTRO'
);

--
CREATE TABLE "Usuario" (
  "id" TEXT PRIMARY KEY,
  "nome" TEXT NOT NULL,
  "email" TEXT UNIQUE NOT NULL,
  "senhaHash" TEXT NOT NULL,
  "role" TEXT NOT NULL,
  "status" "StatusUsuario" NOT NULL DEFAULT 'ATIVO',
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updatedAt" TIMESTAMP(3) NOT NULL
);

CREATE TABLE "Instituicao" (
  "id" TEXT PRIMARY KEY,
  "nome" TEXT UNIQUE NOT NULL,
  "sigla" TEXT,
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updatedAt" TIMESTAMP(3) NOT NULL
);

CREATE TABLE "Aluno" (
  "id" TEXT PRIMARY KEY,
  "usuarioId" TEXT UNIQUE NOT NULL REFERENCES "Usuario"("id") ON DELETE CASCADE,
  "instituicaoId" TEXT NOT NULL REFERENCES "Instituicao"("id"),
  "cpf" TEXT UNIQUE NOT NULL,
  "rg" TEXT NOT NULL,
  "endereco" TEXT NOT NULL,
  "curso" TEXT NOT NULL,
  "saldoMoedas" INTEGER NOT NULL DEFAULT 0,
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updatedAt" TIMESTAMP(3) NOT NULL
);
CREATE INDEX "Aluno_instituicaoId_idx" ON "Aluno"("instituicaoId");

CREATE TABLE "Professor" (
  "id" TEXT PRIMARY KEY,
  "usuarioId" TEXT UNIQUE NOT NULL REFERENCES "Usuario"("id") ON DELETE CASCADE,
  "instituicaoId" TEXT NOT NULL REFERENCES "Instituicao"("id"),
  "cpf" TEXT UNIQUE NOT NULL,
  "departamento" TEXT NOT NULL,
  "saldoMoedas" INTEGER NOT NULL DEFAULT 0,
  "ultimoSemestreCredito" TEXT,
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updatedAt" TIMESTAMP(3) NOT NULL
);
CREATE INDEX "Professor_instituicaoId_idx" ON "Professor"("instituicaoId");

```

```

CREATE TABLE "Empresa" (
  "id" TEXT PRIMARY KEY,
  "usuarioId" TEXT UNIQUE NOT NULL REFERENCES "Usuario"("id") ON DELETE CASCADE,
  "cnpj" TEXT UNIQUE NOT NULL,
  "descricao" TEXT NOT NULL,
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updatedAt" TIMESTAMP(3) NOT NULL
);

CREATE TABLE "Categoria" (
  "id" TEXT PRIMARY KEY,
  "nome" TEXT UNIQUE NOT NULL,
  "slug" TEXT UNIQUE NOT NULL,
  "icone" TEXT
);

CREATE TABLE "Vantagem" (
  "id" TEXT PRIMARY KEY,
  "empresaId" TEXT NOT NULL REFERENCES "Empresa"("id") ON DELETE CASCADE,
  "categoriaId" TEXT REFERENCES "Categoria"("id"),
  "titulo" TEXT NOT NULL,
  "descricao" TEXT NOT NULL,
  "custoMoedas" INTEGER NOT NULL,
  "foto" TEXT,
  "estoque" INTEGER,
  "ativo" BOOLEAN NOT NULL DEFAULT true,
  "validadeCupomDias" INTEGER NOT NULL DEFAULT 30,
  "limitePorAluno" INTEGER,
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updatedAt" TIMESTAMP(3) NOT NULL
);
CREATE INDEX "Vantagem_empresaId_idx" ON "Vantagem"("empresaId");
CREATE INDEX "Vantagem_categoriaId_ativo_idx" ON "Vantagem"("categoriaId", "ativo");

CREATE TABLE "Favorito" (
  "id" TEXT PRIMARY KEY,
  "alunoId" TEXT NOT NULL REFERENCES "Aluno"("id") ON DELETE CASCADE,
  "vantagemId" TEXT NOT NULL REFERENCES "Vantagem"("id") ON DELETE CASCADE,
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE ("alunoId", "vantagemId")
);

CREATE TABLE "CarrinhoItem" (
  "id" TEXT PRIMARY KEY,
  "alunoId" TEXT NOT NULL REFERENCES "Aluno"("id") ON DELETE CASCADE,
  "vantagemId" TEXT NOT NULL REFERENCES "Vantagem"("id") ON DELETE CASCADE,
  "quantidade" INTEGER NOT NULL DEFAULT 1,
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE ("alunoId", "vantagemId")
);

```

```
);
```

```
CREATE TABLE "Reconhecimento" (
  "id" TEXT PRIMARY KEY,
  "professorId" TEXT NOT NULL REFERENCES "Professor"("id") ON DELETE CASCADE,
  "alunoId" TEXT NOT NULL REFERENCES "Aluno"("id") ON DELETE CASCADE,
  "quantidade" INTEGER NOT NULL,
  "mensagem" TEXT NOT NULL,
  "tag" "TagReconhecimento" NOT NULL DEFAULT 'OUTRO',
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX "Reconhecimento_alunoId_createdAt_idx" ON "Reconhecimento"("alunoId",
"createdAt");
CREATE INDEX "Reconhecimento_professorId_createdAt_idx" ON
"Reconhecimento"("professorId", "createdAt");
```

```
CREATE TABLE "Transacao" (
  "id" TEXT PRIMARY KEY,
  "usuarioId" TEXT NOT NULL REFERENCES "Usuario"("id") ON DELETE CASCADE,
  "tipo" "TipoTransacao" NOT NULL,
  "descricao" TEXT NOT NULL,
  "quantidadeMoedas" INTEGER NOT NULL,
  "referenciaId" TEXT,
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX "Transacao_usuarioId_createdAt_idx" ON "Transacao"("usuarioId",
"createdAt");
CREATE INDEX "Transacao_tipo_createdAt_idx" ON "Transacao"("tipo", "createdAt");
```

```
CREATE TABLE "Cupom" (
  "id" TEXT PRIMARY KEY,
  "usuarioId" TEXT NOT NULL REFERENCES "Usuario"("id") ON DELETE CASCADE,
  "vantagemId" TEXT NOT NULL REFERENCES "Vantagem"("id") ON DELETE CASCADE,
  "codigo" TEXT UNIQUE NOT NULL,
  "custoMoedasSnapshot" INTEGER NOT NULL,
  "status" "StatusCupom" NOT NULL DEFAULT 'GERADO',
  "dataValidade" TIMESTAMP(3) NOT NULL,
  "dataUtilizacao" TIMESTAMP(3),
  "validadoPorEmpresaId" TEXT,
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updatedAt" TIMESTAMP(3) NOT NULL
);
CREATE INDEX "Cupom_status_dataValidade_idx" ON "Cupom"("status", "dataValidade");
CREATE INDEX "Cupom_usuarioId_status_idx" ON "Cupom"("usuarioId", "status");
```

```
CREATE TABLE "Notificacao" (
  "id" TEXT PRIMARY KEY,
  "usuarioId" TEXT NOT NULL REFERENCES "Usuario"("id") ON DELETE CASCADE,
  "titulo" TEXT NOT NULL,
  "mensagem" TEXT NOT NULL,
```

```

"lida"          BOOLEAN          NOT          NULL          DEFAULT          false,
"link"          TEXT,
"createdAt"     TIMESTAMP(3)     NOT          NULL          DEFAULT          CURRENT_TIMESTAMP
);
CREATE          INDEX            "Notificacao_usuarioId_lida_createdAt_idx"          ON
"Notificacao"("usuarioId", "lida", "createdAt");

```

4.4 Estratégias de Integridade e Performance

4.4.1 Integridade Referencial

Ação	Comportamento Configurado
Exclusão de Usuario	ON DELETE CASCADE em Aluno, Professor, Empresa, Transacao, Cupom, Notificacao
Exclusão de Empresa	ON DELETE CASCADE em Vantagem (e, por cascata, em Cupons vinculados)
Exclusão de Aluno ou Professor	ON DELETE CASCADE em Reconhecimento
Exclusão de Instituicao	Restrito (não é possível excluir se há alunos/professores vinculados)
Exclusão de Categoria	Permitida; categoriaId em Vantagem fica NULL

4.4.2 Performance

- **Índices estratégicos** em colunas usadas em filtros e ordenações frequentes
- **Índices compostos** otimizam queries multi-coluna (ex: extrato cronológico)
- **Paginação** obrigatória em endpoints de listagem (limit/offset)
- **Conexão poolada** via Prisma (recomendado Prisma Accelerate em produção)
- **Caching** opcional em futuro para dashboards (Redis)

4.4.3 Segurança dos Dados

- **Senhas:** armazenadas apenas como hash bcrypt (salt rounds 10); nunca em texto plano
- **Dados sensíveis** (senhaHash): nunca retornados em responses da API
- **CPF/CNPJ:** exibidos mascarados na UI (111.***.***-11)
- **JWT Secret:** armazenado apenas em variável de ambiente
- **SSL:** conexão com banco PostgreSQL via TLS (padrão Neon)

4.5 Backup e Migrations

4.5.1 Migrations

- Gerenciadas pelo **Prisma Migrate** (prisma/migrations/)
- Versionadas no Git
- Aplicadas em produção via prisma migrate deploy
- Rollback manual (Prisma não oferece rollback automático em produção; estratégia é criar migrations de reversão)

4.5.2 Backup

- **Ambiente de produção (Neon):** backup automático gerenciado pelo provedor, com point-in-time recovery
- **Ambiente local:** pg_dump manual quando necessário
- **Dados críticos:** auditoria via Transacao serve como log imutável de movimentações